

# Unsupervised Machine Learning

Mamta Bhattarai

MBUST

July 7, 2026

# What is Unsupervised Learning?

## Definition

Unsupervised learning is a machine learning technique that learns from **unlabeled data** by discovering hidden structures, relationships, or patterns without knowing the correct output.

## Input

$$X = \{x_1, x_2, \dots, x_n\}$$

No class labels are available.

## Goal

- Discover groups
- Detect anomalies
- Reduce dimensions
- Find associations

# Supervised vs Unsupervised Learning

| Property      | Supervised         | Unsupervised    |
|---------------|--------------------|-----------------|
| Training Data | Labeled            | Unlabeled       |
| Output        | Class/Value        | Hidden Patterns |
| Goal          | Prediction         | Discovery       |
| Examples      | SVM, Decision Tree | K-Means, DBSCAN |

# What is Clustering?

## Idea

Clustering groups similar data points into the same cluster while keeping dissimilar points in different clusters.

Good clustering has

- High similarity within clusters
- Low similarity between clusters

Applications

- Customer segmentation
- Medical diagnosis
- Image segmentation
- Recommendation systems

## K-Means Clustering

### Definition

K-Means is an unsupervised machine learning algorithm that partitions a dataset into  $K$  distinct clusters by grouping similar data points together. Each cluster is represented by its **centroid** (the mean of all points in that cluster).

### Goal

- Maximize similarity within a cluster.
- Minimize similarity between different clusters.
- Find the best centroid locations.

# Intuition Behind K-Means

Imagine a classroom with students having different heights.

Instead of manually grouping students,  
K-Means automatically groups students with similar heights.

## Examples

- Customer segmentation
- Disease subtype identification
- Image compression
- Market basket analysis
- News article clustering

# Working Principle of K-Means

K-Means works by repeatedly refining cluster assignments and centroids.

## Core idea:

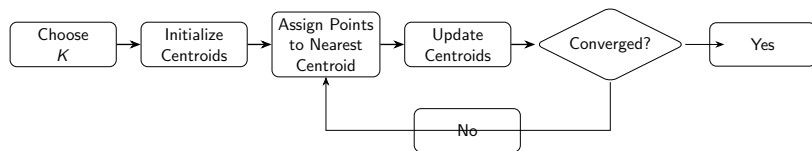
- Assign each data point to the nearest centroid.
- Update each centroid as the mean of assigned points.

Repeat these two steps until centroids stabilize (no significant change).

# K-Means Algorithm

- 1 Choose number of clusters  $K$
- 2 Initialize  $K$  centroids randomly
- 3 Assign each data point to the nearest centroid
- 4 Update centroids using mean of assigned points
- 5 Repeat Steps 3–4 until convergence

# K-Means Algorithm Flow



Algorithm iterates between assignment and update steps until convergence.

# Objective Function

K-Means minimizes the total within-cluster variance.

$$J = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

where

- $K$  = Number of clusters
- $C_i$  = Cluster  $i$
- $\mu_i$  = Centroid of cluster  $i$
- $\|x_j - \mu_i\|^2$  = Squared Euclidean distance

Goal:

$$\min J$$

# Euclidean Distance

K-Means usually uses Euclidean distance.

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}$$

Example

Point A

(2, 3)

Centroid

(5, 7)

Distance

$$\sqrt{(5 - 2)^2 + (7 - 3)^2} = 5$$

The point is assigned to the nearest centroid.

# Example

Suppose we have four points.

$(1, 2), (2, 1), (8, 9), (9, 8)$

Choose

$K = 2$

Initial centroids

$C_1 = (1, 2)$

$C_2 = (9, 8)$

After assigning points,

Cluster 1

$(1, 2), (2, 1)$

Cluster 2

$(8, 9), (9, 8)$

New centroids become

$(1.5, 1.5)$

and

$(8.5, 8.5)$

# K-Means: One Iteration (Distance + Update)

**Data:**

$$(1, 1), (2, 1), (2, 2), (8, 8), (9, 8), (8, 9), \quad K = 2$$

**Init:**

$$\mu_1 = (1, 1), \quad \mu_2 = (9, 8)$$

**Centroid Update**

$$\mu_1 = (1.67, 1.33)$$

$$\mu_2 = (8.33, 8.33)$$

**Distance Table**

| Point | $d(\mu_1)$ | Cluster |
|-------|------------|---------|
| (1,1) | 0.00       | 1       |
| (2,1) | 1.00       | 1       |
| (2,2) | 1.41       | 1       |
| (8,8) | 9.90       | 2       |
| (9,8) | 10.63      | 2       |
| (8,9) | 10.63      | 2       |

**Result**

$$C_1 = \{(1, 1), (2, 1), (2, 2)\}$$

$$C_2 = \{(8, 8), (9, 8), (8, 9)\}$$

**Stop: No change**

# Choosing the Number of Clusters

Choosing an appropriate number of clusters ( $K$ ) is one of the most important steps in K-Means clustering.

## If $K$ is too small

- Different groups are merged together.
- Clusters become less meaningful.
- High within-cluster variation.

## If $K$ is too large

- One natural cluster is split into many small clusters.
- Overfitting may occur.
- Interpretation becomes difficult.

## Common Methods for Selecting $K$

Elbow Method

Silhouette Score

Gap Statistic

# Methods for Choosing the Optimal Number of Clusters

| Method                  | Idea                                                                                              | Selection Rule                                                    |
|-------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Elbow Method</b>     | Monitor the decrease in Within-Cluster Sum of Squares (WCSS) as the number of clusters increases. | Choose the <b>elbow point</b> where improvement starts to slow.   |
| <b>Silhouette Score</b> | Measures cluster compactness and separation.                                                      | Choose the $K$ with the <b>highest average silhouette score</b> . |
| <b>Gap Statistic</b>    | Compares clustering quality with randomly generated data.                                         | Choose the $K$ with the <b>largest Gap value</b> .                |

## Observation

All three methods attempt to identify the value of  $K$  that produces compact, well-separated, and meaningful clusters.

# Comparison of Methods

## Elbow Method

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

### Symbols:

- $K$  = number of clusters
- $C_k$  = cluster  $k$
- $x_i$  = data point
- $\mu_k$  = centroid (mean) of cluster  $k$
- $\|x_i - \mu_k\|^2$  = squared distance

## Silhouette Score

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

### Symbols:

- $s(i)$  = silhouette value for point  $i$
- $a(i)$  = average distance within same cluster
- $b(i)$  = average distance to nearest other cluster
- $\max$  = maximum of two values

## Gap Statistic

$$Gap(K) = \mathbb{E}[\log(W_k^*)] - \log(W_k)$$

### Symbols:

- $K$  = number of clusters
- $W_k$  = within-cluster variation
- $W_k^*$  = variation from random data
- $\mathbb{E}$  = expectation (average value)
- $\log$  = logarithm function

| Method        | Speed  | Accuracy  |
|---------------|--------|-----------|
| Elbow         | High   | Medium    |
| Silhouette    | Medium | High      |
| Gap Statistic | Low    | Very High |

## Advantages

- Simple
- Fast
- Easy to implement
- Works well on large datasets
- Scalable

## Limitations

- Need to specify  $K$
- Sensitive to initialization
- Sensitive to outliers
- Assumes spherical clusters
- May converge to local optimum

# Applications of K-Means

K-Means is widely used in many fields.

- Customer segmentation
- Image segmentation
- Document clustering
- Recommendation systems
- Medical diagnosis
- Gene expression analysis
- Network traffic analysis
- Market research

## Key Points

- K-Means is an unsupervised clustering algorithm.
- It partitions data into  $K$  clusters.
- Each cluster is represented by its centroid.
- The algorithm iteratively updates assignments and centroids.
- It minimizes the within-cluster sum of squared distances.
- It is simple, efficient, and widely used.

# Hierarchical Clustering

**Hierarchical Clustering** is an **unsupervised learning** technique that groups similar data points into a hierarchy of nested clusters.

- Builds clusters based on the similarity (or distance) between data points.
- Unlike K-Means, it **does not require specifying the number of clusters** in advance.
- The clustering process is represented using a **dendrogram**, a tree-like diagram that shows how clusters are formed.
- The final number of clusters is obtained by cutting the dendrogram at an appropriate height.

## Types of Hierarchical Clustering

- **Agglomerative (Bottom-Up)**: Starts with individual data points and repeatedly merges the closest clusters.
- **Divisive (Top-Down)**: Starts with one large cluster and recursively splits it into smaller clusters.

# Agglomerative Hierarchical Clustering

**Agglomerative Clustering** is the most commonly used hierarchical clustering algorithm.

## Algorithm

- 1 Treat each data point as a separate cluster.
- 2 Compute the distance between every pair of clusters.
- 3 Merge the two closest clusters.
- 4 Update the distance matrix.
- 5 Repeat until all points form a single cluster.

## Common Linkage Criteria

- **Single Linkage:** Minimum distance between two clusters.
- **Complete Linkage:** Maximum distance between two clusters.
- **Average Linkage:** Average distance between all pairs of points.
- **Ward's Linkage:** Minimizes the increase in within-cluster variance.

# Dendrogram

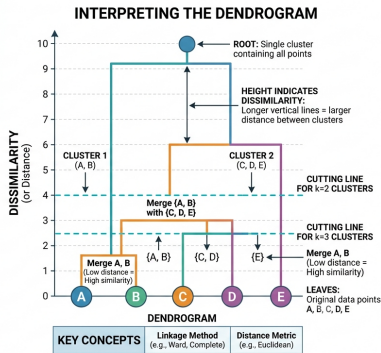
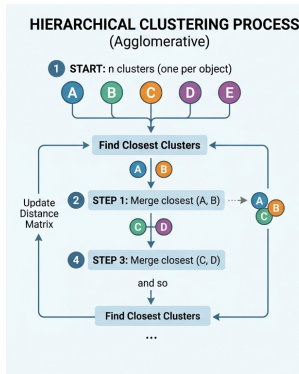
A **dendrogram** is a tree-like diagram that shows how clusters are formed during hierarchical clustering.

- Each **leaf (bottom node)** represents an individual data point.
- Each **branch** represents the merging of two clusters.
- The **height of the merge** indicates the distance (or dissimilarity) between the merged clusters.
- Clusters that merge at a lower height are more similar than those merging at a higher height.
- A horizontal cut through the dendrogram determines the final number of clusters.

## Key Idea

The dendrogram helps visualize the clustering process and choose an appropriate number of clusters.

# Reading a Dendrogram



- Start reading from the bottom, where each object is a separate cluster.
- Move upward to observe how clusters are merged.
- The vertical axis represents the distance (linkage value) at which clusters join.
- Draw a horizontal line to decide the desired number of clusters.

# Example: Distance Matrix

|          | <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>E</i> |
|----------|----------|----------|----------|----------|----------|
| <i>A</i> | 0        | 1        | 4        | 7        | 8        |
| <i>B</i> | 1        | 0        | 3        | 6        | 7        |
| <i>C</i> | 4        | 3        | 0        | 3        | 4        |
| <i>D</i> | 7        | 6        | 3        | 0        | 1        |
| <i>E</i> | 8        | 7        | 4        | 1        | 0        |

# Step 1: Merge A and B

**Smallest distance = 1**

$$(A, B) \rightarrow AB$$

**Single Linkage Formula**

$$d(AB, X) = \min\{d(A, X), d(B, X)\}$$

**Compute the new distances**

$$d(AB, C) = \min(4, 3) = 3, d(AB, D) = \min(7, 6) = 6,$$

$$d(AB, E) = \min(8, 7) = 7$$

**Updated Distance Matrix**

|           | <i>AB</i> | <i>C</i> | <i>D</i> | <i>E</i> |
|-----------|-----------|----------|----------|----------|
| <i>AB</i> | 0         | 3        | 6        | 7        |
| <i>C</i>  | 3         | 0        | 3        | 4        |
| <i>D</i>  | 6         | 3        | 0        | 1        |
| <i>E</i>  | 7         | 4        | 1        | 0        |

## Step 2: Merge D and E

**Smallest distance = 1**

$$(D, E) \rightarrow DE$$

**Single Linkage Formula**

$$d(DE, X) = \min\{d(D, X), d(E, X)\}$$

**Compute the new distances**

$$d(DE, AB) = \min(6, 7) = 6$$

$$d(DE, C) = \min(3, 4) = 3$$

**Updated Distance Matrix**

|           | <i>AB</i> | <i>C</i> | <i>DE</i> |
|-----------|-----------|----------|-----------|
| <i>AB</i> | 0         | 3        | 6         |
| <i>C</i>  | 3         | 0        | 3         |
| <i>DE</i> | 6         | 3        | 0         |

## Step 3: Merge AB and C

**Smallest distance = 3**

$$(AB, C) \rightarrow ABC$$

**Single Linkage Formula**

$$d(ABC, DE) = \min\{d(AB, DE), d(C, DE)\}$$

**Compute the new distance**

$$d(ABC, DE) = \min(6, 3) = 3$$

**Updated Distance Matrix**

|     | ABC | DE |
|-----|-----|----|
| ABC | 0   | 3  |
| DE  | 3   | 0  |

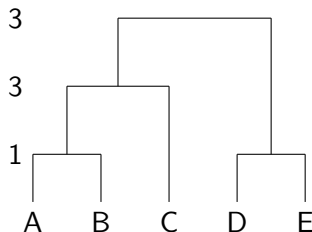
## Step 4: Final Merge

**Smallest distance = 3**

$$(ABC, DE) \rightarrow ABCDE$$

$$\boxed{\{A, B, C, D, E\}}$$

# Dendrogram Example



The dendrogram shows how clusters are merged at different distances.

# Advantages and Limitations of Hierarchical Clustering

## Advantages

- No need to specify the number of clusters initially.
- Produces an interpretable dendrogram.
- Can discover clusters of different sizes.
- Works well for small and medium-sized datasets.
- Applicable with various distance and linkage measures.

## Limitations

- Computationally expensive for large datasets.
- Sensitive to noise and outliers.
- Once clusters are merged or split, the decision cannot be reversed.
- Performance depends on the chosen linkage method.

## Key Point

Hierarchical clustering provides a complete hierarchy of clusters, whereas K-Means directly partitions data into a predefined number of clusters.

**DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is a density-based clustering algorithm.

**Main idea:** Clusters are formed in regions where data points are densely packed, while sparse regions are treated as noise.

**Key capabilities:**

- Finds clusters of arbitrary shape
- Automatically detects outliers (noise)
- Does not require number of clusters  $K$

# Key Parameters of DBSCAN

## $\epsilon$ (Epsilon)

- Defines the radius of neighborhood around a point
- Two points are neighbors if distance  $\leq \epsilon$

## MinPts

- Minimum number of points required inside  $\epsilon$ -neighborhood
- Typically: MinPts  $\geq$  dimensionality + 1

## Distance metric:

$$d(x_i, x_j) = \sqrt{(x_i - x_j)^2}$$

# Point Types in DBSCAN

## 1. Core Point

- Has at least  $\text{MinPts}$  within  $\epsilon$

## 2. Border Point

- Has fewer than  $\text{MinPts}$  neighbors
- Lies within  $\epsilon$  of a core point

## 3. Noise Point

- Not reachable from any core point
- Treated as outlier

# DBSCAN Example Dataset

Consider the following 2D points:

$(1, 1), (1.2, 1.1), (1.3, 0.9), (5, 5), (5.1, 5.2), (4.9, 4.8), (8, 1)$

Assume:

$$\varepsilon = 0.5, \quad \text{MinPts} = 3$$

**Goal:** Identify dense regions and noise.

# Neighborhood Check (Core Point Detection)

## Check point (1,1):

Neighbors within  $\varepsilon = 0.5$ :

- (1.2,1.1)
- (1.3,0.9)

$\Rightarrow$  Total points = 3 (including itself)

**So (1,1) is a CORE point.**

# Cluster Formation

**Cluster 1: Dense region near (1,1)**

$(1, 1), (1.2, 1.1), (1.3, 0.9)$

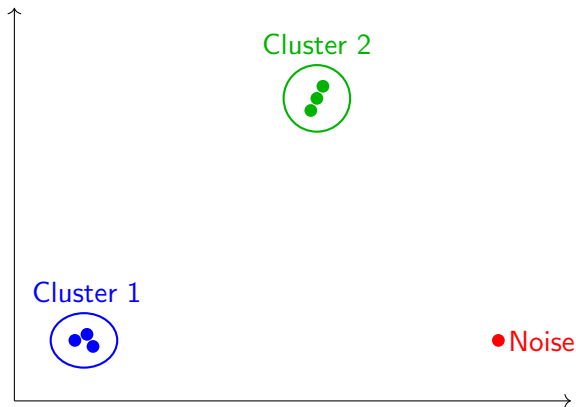
**Cluster 2: Dense region near (5,5)**

$(5, 5), (5.1, 5.2), (4.9, 4.8)$

**Noise point:**

$(8, 1)$

# Final Cluster Formation



Cluster 1 =  $\{(1, 1), (1.2, 1.1), (1.3, 0.9)\}$

Cluster 2 =  $\{(5, 5), (5.1, 5.2), (4.9, 4.8)\}$

Noise =  $\{(8, 1)\}$

# DBSCAN Working Principle

- Start from an unvisited point
- Find all neighbors within  $\varepsilon$
- If neighbors  $\geq \text{MinPts}$   $\rightarrow$  form a cluster
- Expand cluster by recursively visiting neighbors
- Otherwise mark as noise or border

**Key idea:** Clusters grow based on density connectivity.

# Advantages and Limitations of DBSCAN

## Advantages

- Detects arbitrary-shaped clusters
- Robust to noise and outliers
- No need to predefine number of clusters

## Limitations

- Sensitive to  $\varepsilon$  and MinPts
- Poor performance with varying density clusters
- Struggles in high-dimensional data

# What is Anomaly Detection?

**Anomaly Detection** (also called **Outlier Detection**) is the process of identifying observations that deviate significantly from the normal behavior of a dataset.

Anomalies are rare events that may indicate:

- Fraudulent activities
- System failures
- Security attacks
- Measurement errors
- Rare but important events

## Real-world Applications

- Credit card fraud detection
- Network intrusion detection
- Medical diagnosis (abnormal test results)
- Manufacturing defect detection
- Sensor monitoring in IoT systems

# Types of Anomalies

There are three major types of anomalies.

## 1. Point Anomaly

A single observation is significantly different from the rest.

Example: A credit card transaction of \$10,000 when the customer's usual spending is below \$100.

## 2. Contextual Anomaly

A data point is abnormal only in a specific context such as time or location.

Example: A temperature of 30°C is normal in summer but abnormal in winter.

## 3. Collective Anomaly

A group of observations together forms an abnormal pattern, even if individual points appear normal.

Example: A sudden burst of network requests indicating a cyber attack.

# Common Approaches for Anomaly Detection

Different techniques detect anomalies using different assumptions about the data.

- **Statistical Methods**

- Assume a probability distribution.
- Values with very low probability are considered anomalies.

- **Distance-Based Methods**

- Points far away from their nearest neighbors are treated as outliers.

- **Density-Based Methods**

- Compare the local density of a point with that of its neighbors.

- **Clustering-Based Methods**

- Points that do not belong to any cluster or belong to very small clusters are considered anomalies.

# Machine Learning Methods for Anomaly Detection

Machine learning techniques learn the normal behavior of the data and identify observations that deviate from it.

| Method                     | Idea                                                                                                         |
|----------------------------|--------------------------------------------------------------------------------------------------------------|
| Isolation Forest           | Randomly isolates anomalies using decision trees. Outliers require fewer splits to isolate.                  |
| One-Class SVM              | Learns the boundary of normal data and flags observations outside the boundary.                              |
| Local Outlier Factor (LOF) | Compares the local density of a point with the density of its neighbors. Lower density indicates an outlier. |
| Autoencoder                | Neural network reconstructs normal data accurately. Large reconstruction error indicates an anomaly.         |

# Summary

- Anomaly detection identifies unusual or rare observations.
- Anomalies may indicate fraud, faults, attacks, or abnormal events.
- Three main types:
  - Point anomalies
  - Contextual anomalies
  - Collective anomalies
- Detection methods include statistical, distance-based, density-based, clustering-based, and machine learning techniques.
- Choosing an appropriate method depends on the data characteristics and application.

# Comparison of Clustering Algorithms

| Property          | K-Means | Hierarchical | DBSCAN   |
|-------------------|---------|--------------|----------|
| Need K beforehand | Yes     | No           | No       |
| Handles Outliers  | Poor    | Moderate     | Good     |
| Arbitrary Shapes  | No      | Limited      | Yes      |
| Scalability       | High    | Low          | Moderate |
| Noise Detection   | No      | No           | Yes      |

# Summary

- Unsupervised learning works with unlabeled data.
- K-Means partitions data into  $K$  clusters.
- Hierarchical clustering creates a dendrogram of nested clusters.
- DBSCAN discovers density-based clusters and detects noise.
- Anomaly detection identifies unusual patterns and outliers.